# Project Guide

# Entropy-Guided Hierarchical Load Balancing for Heterogeneous HPC Workloads

## 1. Purpose Of The Project

This project builds an intelligent load balancing and job scheduling system for a High-Performance Computing cluster.

An HPC cluster contains many compute nodes. These nodes may have different CPU, memory, network, and disk capacities. Jobs arrive over time, and the scheduler must decide which node should run each job.

The purpose of this project is to show that scheduling can be improved by considering more than just current CPU usage. The project considers:

- Current CPU, memory, network, and disk usage
- Node hardware capacity
- Historical node load behavior
- Node stability using entropy
- Job type and resource requirements
- Cluster hierarchy, such as racks and nodes

The final goal is to compare the proposed scheduler with traditional algorithms and measure whether it improves fairness, load balance, waiting time, and scheduling quality.

## 2. Why This Method Is Used

Traditional schedulers often use simple rules:

- Round Robin
- Random assignment
- Least loaded node
- First available node

These methods are easy to implement, but they can make poor decisions.

For example, a node may have low CPU usage but very high memory or disk usage. A CPU-only scheduler may choose that node even though it is not suitable for a memory-heavy or I/O-heavy job.

Another problem is stability. Two nodes may currently show the same CPU usage, but their past behavior may be very different:

```
Node A: 40, 42, 41, 40, 43
Node B: 10, 90, 20, 85, 15
```

Node A is stable. Node B is unpredictable. A long-running important job should usually prefer Node A.

This project uses entropy to measure how stable or unstable a node is. Low entropy means stable. High entropy means highly variable.

The proposed scheduler combines:

- Multi-resource load scoring
- Entropy-based stability classification
- Job-type-aware resource weights
- Hierarchical rack/node search

This makes the scheduling decision more informed.

## 3. Use Of The Project

This project can be used for:

- Research on HPC job scheduling
- Comparing load balancing algorithms
- Simulating heterogeneous clusters
- Studying node stability and workload behavior
- Creating experiment results for academic reports
- Demonstrating entropy-guided scheduling
- Building a dashboard for cluster monitoring

It is not controlling a real HPC cluster yet. It is currently a simulator. That makes it safe and useful for experimentation.

## 4. How The Project Works

The system follows this workflow:

```
Job arrives
-> Job is classified
-> Resource weights are selected
-> Cluster hierarchy is checked
-> Racks are evaluated
-> Candidate nodes are filtered
-> Load score is calculated
-> Entropy penalty is calculated
-> Best node is selected
-> Job is assigned
-> Node, rack, and cluster metrics are updated
-> Results are collected
```

The cluster has a hierarchy:

```
Cluster
|-- Rack 1
|   |-- Node 1
|   |-- Node 2
|-- Rack 2
|   |-- Node 3
|   |-- Node 4
```

Each node stores:

- CPU capacity
- Memory capacity
- Network capacity
- Disk capacity
- Current utilization
- Historical utilization
- Entropy score
- Entropy class
- Active jobs

The scheduler calculates a composite load score:

```
Composite Load =
CPU weight * CPU usage
+ Memory weight * Memory usage
+ Network weight * Network usage
+ Disk weight * Disk usage
```

Weights depend on the job type.

For a CPU-bound job:

```
CPU: 0.60
Memory: 0.15
Network: 0.15
Disk: 0.10
```

For a memory-bound job:

```
CPU: 0.15
Memory: 0.60
Network: 0.15
Disk: 0.10
```

For an I/O-bound job:

```
CPU: 0.15
Memory: 0.15
Network: 0.35
Disk: 0.35
```

The final scheduling score is:

```
Final Score =
Composite Load Score
+ Entropy Penalty
+ Resource Pressure Penalty
+ Priority Adjustment
```

The node with the lowest final score is selected.

# 5. Entropy In This Project

Entropy is used to understand node stability.

The project supports:

- Variance-based entropy
- Shannon entropy

Variance-based entropy is used by default because it is simple and effective for the first version.

Low entropy means the node has stable historical usage.

Medium entropy means the node has moderate variation.

High entropy means the node is unstable or bursty.

The scheduler prefers low-entropy nodes for:

- Long-running jobs
- Predictable jobs
- High-priority jobs

High-entropy nodes can still be used for:

- Short jobs
- Flexible jobs
- Bursty jobs

# 6. Dataset Used

The project can run in two modes:

1. Synthetic workload mode
2. Real dataset mode

The current real dataset folder is:

```
task_clustering_batch_datasets/
```

The project currently loads this real HPC workload trace by default:

```
task_clustering_batch_datasets/workloads/hpc2n.swf
```

This is an SWF, or Standard Workload Format, trace. It contains real batch scheduling data such as:

- Job ID
- Submit time
- Wait time
- Runtime
- Allocated processors
- Requested processors
- Requested time
- Requested memory
- User ID
- Queue
- Partition

SWF traces usually do not contain direct network and disk utilization. For that reason, this project maps CPU and memory from real fields and estimates network and disk requirements from runtime and memory pressure.

If the real dataset folder is missing, the project falls back to synthetic job generation.

# 7. Algorithms Implemented

The project compares these algorithms:

- Round Robin
- Random Scheduler
- Least Loaded Scheduler
- Entropy-Guided Hierarchical Scheduler

Round Robin assigns jobs in node order.

Random Scheduler selects a random node that can accept the job.

Least Loaded Scheduler selects the node with the lowest current load.

Entropy-Guided Hierarchical Scheduler first evaluates racks, then nodes, and uses resource load, entropy, resource pressure, and job priority.

# 8. Performance Metrics

The project calculates:

- Completed jobs
- Rejected jobs
- Makespan
- Average waiting time
- 95th percentile waiting time
- Scheduling overhead
- Load variance
- Load standard deviation
- Jain's fairness index
- Average CPU utilization
- Average memory utilization
- Average network utilization
- Average disk utilization

Jain's fairness index measures how evenly load is distributed. A value closer to 1.0 means better fairness.

# 9. Technology Stack

Main language:

```
Python
```

Core libraries:

```
NumPy
Pandas
SimPy
Plotly
Matplotlib
Streamlit
Pytest
```

Current project modules:

```
cluster/        Cluster, rack, and node models
jobs/           Job model, generator, classifier, trace parser
entropy/        Entropy calculation and classification
scheduler/      Scheduling algorithms
simulation/     Metric and job simulation
evaluation/     Metrics and experiments
visualization/  Dashboard and graphs
config/         Project settings
results/        Generated experiment outputs
data/           Data storage folders
tests/          Unit tests
```

# 10. Project Requirements

Software requirements:

- Python 3.10 or newer
- pip
- Virtual environment support

Python package requirements are listed in:

```
requirements.txt
```

Install them with:

```
pip install -r requirements.txt
```

Dataset requirement:

- No dataset is strictly required because synthetic workload generation is available.

- For real-trace experiments, keep the dataset folder in the project root:

```
task_clustering_batch_datasets/
```

## 11. How To Run The Project

Open a terminal in the project folder:

```
cd C:\Users\sruja\Documents\project
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it on Windows:

```
.venv\Scripts\activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the main simulation:

```
python main.py
```

Run tests:

```
python -m pytest
```

Run the Streamlit dashboard:

```
python -m streamlit run visualization/dashboard.py --server.address 127.0.0.1 --server.port 8501
```

Then open:

```
http://127.0.0.1:8501
```

## 12. Output Files

When the real HPC2N dataset is available, the project writes:

```
results/real_hpc2n_algorithm_comparison.csv
results/real_hpc2n_algorithm_comparison.json
results/real_hpc2n_algorithm_comparison.txt
```

These files contain algorithm comparison results.

If the dataset is missing and synthetic mode is used, result files use a synthetic output prefix.

## 13. Current Status

The current version is a complete working v1 simulator.

Implemented:

- Cluster model
- Metric simulation
- Entropy calculation
- Job generation
- Real SWF trace parsing
- Baseline schedulers
- Proposed entropy-guided scheduler
- Simulation engine
- Evaluation metrics
- Result export
- Streamlit dashboard
- Tests

Future improvements can include:

- More realistic queue behavior
- Larger cluster scaling experiments
- More detailed workflow DAG support from `.dax` files
- Better resource inference from real traces
- Machine learning-based load prediction
- More advanced visualization pages